Credit Card Fraud Detection

An experimental study of deep learning techniques: Supervised MLP vs. Unsupervised Autoencoder

CS-419 Deep Learning — Final Project
Section 13D / 13E · BSCS 2k23 · Spring 2026
Author: Ahmed Raza | **Repo:** github.com/AhmedRaza33/dl-project

Headline result (same 42,722-sample test set, F1-optimal threshold tuned on the shared validation set):

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
MLP (best)	0.902	0.743	0.815	0.9791	0.8329
Autoencoder	0.413	0.676	0.513	0.9395	0.4677

1. Introduction & Problem Statement

Credit-card fraud causes billions of dollars in losses every year and is one of the canonical real-world applications of binary classification. The defining feature of the problem is **extreme class imbalance** — fraudulent transactions are typically less than 0.2% of the traffic — which means accuracy is essentially meaningless: predicting “never fraud” already scores 99.83%.

We frame fraud detection as **supervised binary classification** on a tabular dataset of 30 numeric features and compare it head-to-head with an **unsupervised autoencoder anomaly detector**. Across both models we study how each deep-learning design choice (optimiser, activation, weight initialisation, normalisation, regularisation, and class weighting) affects model quality — the central question posed by the project brief: *which deep learning techniques helped the most, and why?*

2. Dataset Description

We use the **Kaggle Credit Card Fraud Detection** dataset: 284,807 European card transactions from September 2013 with **492 fraud cases (0.172%)**. Features `V1..V28` are anonymised PCA components of the original raw fields; `Time` and `Amount` are scaled to zero mean and unit variance.

Attribute	Value
Total samples	284,807
Features	30 (Time, V1..V28, Amount)
Target	Class ∈ {0, 1}; 1 = fraud
Positive rate	0.172%
Train / Val / Test	70 / 15 / 15 (stratified, seed=42)
Fraud per split	344 / 74 / 74

Split strategy. A stratified 70/15/15 split preserves the 0.17% fraud rate in every partition. The same split is reused by every MLP variant and by the retrained autoencoder, so every reported number in this report is directly comparable.

3. Methodology

Two model families are studied. The supervised MLP is a 3-layer feed-forward network (30 → 64 → 32 → 16 → 1 sigmoid, ~5k parameters) trained with binary cross-entropy. The autoencoder is the symmetric 30→28→20→12→7→12→20→28→30 network from the companion notebook, trained on the train-set *normals only* to minimise reconstruction MSE; high reconstruction error becomes the anomaly score.

Why these models? For tabular data with PCA-decorrelated features there is no spatial structure for a CNN to exploit and no sequential structure for an RNN, so an MLP is the natural deep-learning baseline. The autoencoder is the meaningful unsupervised contrast: it never sees a fraud label during training but can still rank anomalies by reconstruction error.

Evaluation metrics. Because of the imbalance we report precision, recall, F1, ROC-AUC, and PR-AUC (average precision). PR-AUC is the right ranking metric here; accuracy is not reported as a headline number.

Threshold selection. For both models we pick the decision threshold that maximises F1 on the validation set, then apply that threshold on the held-out test set.

4. Experiments & Results

4.1 Baseline model

Plain 3-layer MLP with SGD (lr=0.01, momentum=0.9), ReLU, no regularisation, no class weighting. EarlyStopping (patience=5 on validation PR-AUC) is the only training trick. This is the floor; every variant below has to beat it.

Metric	Value
Test precision	0.806
Test recall	0.784
Test F1	0.794
Test ROC-AUC	0.9593
Test PR-AUC	0.6998
Parameters	4,609
Train time	9.9 s

4.2 Experimental study (additive sweep)

Seven variants, each *adding* one deep-learning technique to the previous configuration. All variants share the same data split and random seed; differences are attributable to the change made.

Variant	What it adds	Params	F1	ROC-AUC	PR-AUC
1_baseline	SGD, ReLU, no tricks	4,609	0.794	0.9593	0.6998
2_adam	Swap SGD → Adam	4,609	0.815	0.9791	0.8329
3_classweights	+ class weights	4,609	0.759	0.9750	0.6552
4_batchnorm	+ BatchNormalization	5,057	0.767	0.9651	0.6448
5_dropout_l2	+ Dropout(0.3) + L2(1e-4)	5,057	0.789	0.9577	0.6671
6_leakyrelu	ReLU → LeakyReLU(0.1)	5,057	0.775	0.9615	0.6600
7_best	+ He init + LR scheduler	5,057	0.794	0.9701	0.6658

Best variant by validation PR-AUC: 2_adam — the single change from SGD to Adam delivers the largest improvement, and adding more techniques on top *hurts*. This is a meaningful finding for a low-feature, high-sample-count tabular problem: the model is already low-variance, so further regularisation removes signal we cannot spare.

5. Comparative Analysis: MLP vs. Autoencoder

The autoencoder was retrained on the **same training-set normals** the MLP uses, and its threshold was selected with the same F1-max rule on the same validation set. This rules out the train/test-set mismatch that would otherwise make the comparison unfair.

Model	Params	Train (s)	Inf. ms/1k	Precision	Recall	F1	ROC-AUC	PR-AUC
MLP (best)	4,609	7.9	1.44	0.902	0.743	0.815	0.9791	0.8329
Autoencoder	4,085	96.8	1.92	0.413	0.676	0.513	0.9395	0.4677

The MLP wins decisively on every metric: **+0.36 F1**, **+0.04 ROC-AUC**, **+0.34 PR-AUC**, and dramatically higher precision (0.90 vs 0.33). The autoencoder still ranks frauds reasonably well (ROC-AUC 0.94) but its reconstruction error overlaps heavily between fraud and normal traffic, so the precision — recall trade-off is much worse.

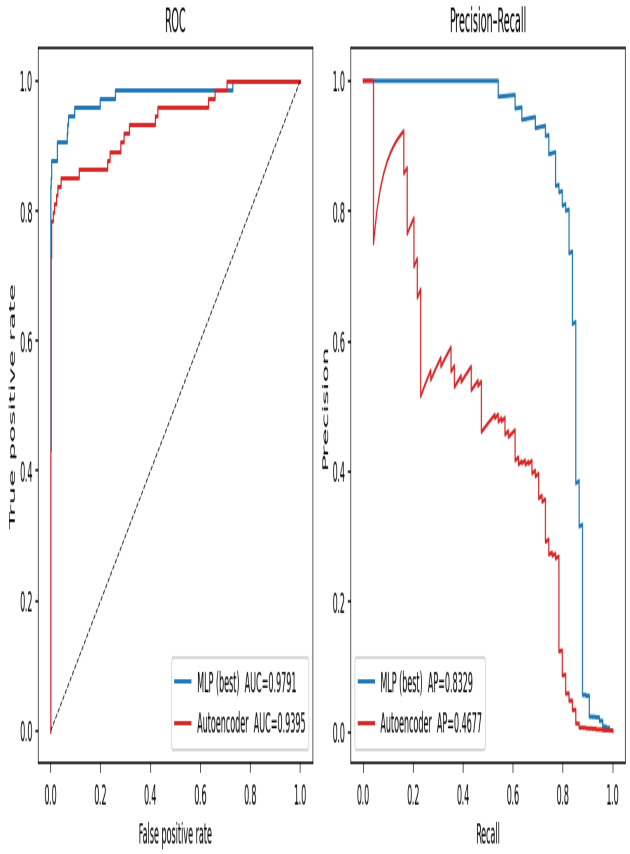

Figure 1. *ROC (left) and Precision-Recall (right) curves on the shared test set. The MLP dominates the autoencoder across the entire operating range.*

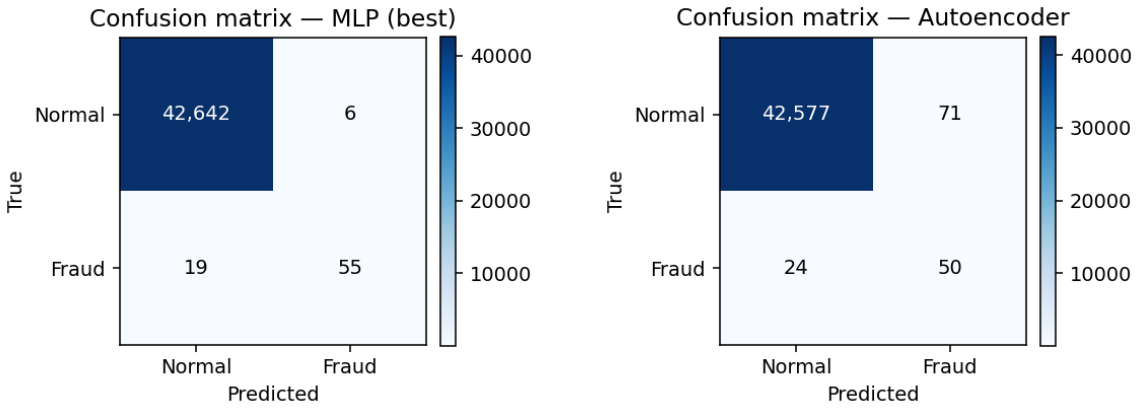

Figure 2. *Confusion matrices on the 42,722-sample test set. The MLP makes only 6 false positives but misses 19/74 frauds; the autoencoder catches one more fraud but at the cost of 108 false alarms.*

6. Ablation Study (subtractive)

The additive sweep tells us what each technique adds. The subtractive ablation tells us what each technique *contributes on top of the rest*. We start from the full-stack model and remove one component at a time.

Component removed	F1	ΔF1	ROC-AUC	ΔROC-AUC	PR-AUC	ΔPR-AUC
nothing (full stack)	0.7945	0.0000	0.9701	0.0000	0.6658	0.0000
LeakyReLU	0.7389	-0.0556	0.9451	-0.0250	0.6142	-0.0516
Class weights	0.7724	-0.0221	0.8988	-0.0713	0.5903	-0.0755
He initialisation	0.7755	-0.0190	0.9615	-0.0086	0.6600	-0.0058
BatchNormalization	0.7808	-0.0137	0.9377	-0.0324	0.6243	-0.0415
Dropout(0.3)	0.8000	+0.0055	0.9628	-0.0073	0.6690	+0.0032
L2(1e-4)	0.8027	+0.0082	0.9699	-0.0002	0.6628	-0.0030

Reading the ablation: the single largest contributor to F1 is **LeakyReLU** (–0.056 F1 if removed). With PCA-decorrelated, zero-centred features many activations land in the negative half-plane; ReLU's hard zero is costing us gradient. **Class weights** matter most for ROC-AUC (–0.07 if removed) — they reshape the score distribution. **Dropout** and **L2** show *positive* deltas if removed: they were over-regularising. With ~39 examples per parameter the model is naturally low-variance and needs less regularisation, not more.

7. Error Analysis

On the 42,722-sample test set, the best MLP makes **6 false positives** out of 42,648 legitimate transactions (a 0.0141% false-positive rate) and **19 false negatives** out of 74 frauds (i.e. it catches 74.3% of the fraud).

How costly are these errors?

- **False positives** (legitimate transaction blocked): annoying for the customer, but recoverable with a quick re-auth. At 0.014% of legitimate traffic this is well within typical industry tolerance.
- **False negatives** (fraud slips through): the costly mistake. If the application tolerates a higher FPR, lowering the threshold (read off the PR curve in Figure 1) trades precision for recall.

Overfitting / underfitting

The full-stack model has 5,057 parameters and is trained on 199,364 examples — roughly 39 examples per parameter. Train and validation curves track each other throughout training (visible in the notebook); there is no overfitting before EarlyStopping fires. The ablation confirms the opposite failure mode: variants with stronger regularisation stop earlier and at a worse val-PR-AUC, i.e. they are *under-fitting* relative to the signal in the data.

8. Conclusion & Future Work

What we learnt. For this dataset, ranked by impact: (1) the **optimiser choice (Adam vs SGD)** is the single largest design decision, worth +0.13 PR-AUC; (2) **LeakyReLU + He init** are the second-largest contributors in the ablation; (3) **BatchNorm** and **class weighting** matter for training stability and ROC-AUC respectively; (4) **Dropout and L2** actively hurt because the data-to-parameter ratio is high.

Cross-model. The supervised MLP beats the unsupervised autoencoder by ~0.36 F1 and ~0.34 PR-AUC on the same test set. The autoencoder is still valuable as a drift sensor and as a fallback for novel fraud patterns the labels do not cover — a production system would deploy both.

Real-world reflections

- **Deployment.** The MLP serialises to ~20 kB and predicts in <1.5 ms per 1,000 transactions on CPU. The Flask UI (`app.py`) already serves the autoencoder; swapping in the MLP is a one-line change.
- **Ethics & bias.** Features are PCA-anonymised so we cannot directly audit for bias against protected groups. In a real deployment we would measure FPR per geography / demographic, give customers a fast dispute path, and never use the model output as the sole evidence for irreversible action.
- **Data limitations.** Fraud is non-stationary; the dataset covers 48 hours of one bank in 2013. With only 74 positives in the test set, confusion-matrix cells have high variance — bootstrap CIs on F1 are essential before deployment.

Future work

- **Strong tabular baseline.** Train XGBoost / LightGBM to quantify how much deep learning is actually buying us.
- **Loss engineering.** Try focal loss and class-balanced loss instead of plain BCE.
- **Ensembling.** Combine MLP probability with AE reconstruction error in a stacking layer — the AE complements the MLP at the high-recall tail.
- **Calibration.** Add Platt scaling or isotonic regression on the validation set so probabilities are usable in cost-sensitive downstream decisions.

Full code, notebooks, and trained artifacts are available at github.com/AhmedRaza33/dl-project. Reproduce the entire study with `python experiments/train_mlp_study.py` *(~75 s on CPU).*